CS471 - Web Technologies

Lab 1 Report - Wireshark Network Traffic Analysis

Leena Mishal ALmutairi
431201604

5532

Part 1: Capturing HTTP Traffic

Task 1: Start Wireshark and Capture Packets

Wireshark was opened and the Wi-Fi interface connected to the internet was selected. Packet capturing was started using the "Start Capturing Packets" (shark fin) button. The browser was then used to navigate to http://neverssl.com/, an unencrypted HTTP website. After the page finished loading, the capture was stopped using the red stop button.

Task 2: Filter HTTP Packets and Analyze Them

In the filter bar, "http" was typed and applied, which filtered out only the HTTP packets from the capture. An HTTP packet was then selected to view its details, and the request and response messages were observed.

HTTP Request

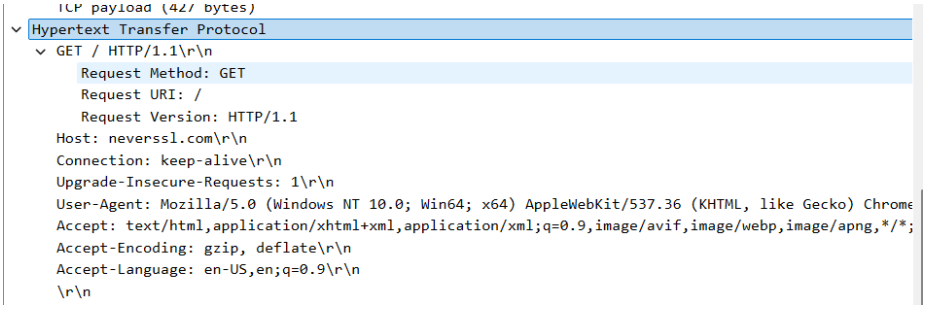


Figure 1: HTTP GET request to neverssl.com

Method: GET Request URI: / Version: HTTP/1.1 Host: neverssl.com

The browser sent a GET request for the root page of neverssl.com using HTTP/1.1.

HTTP Response

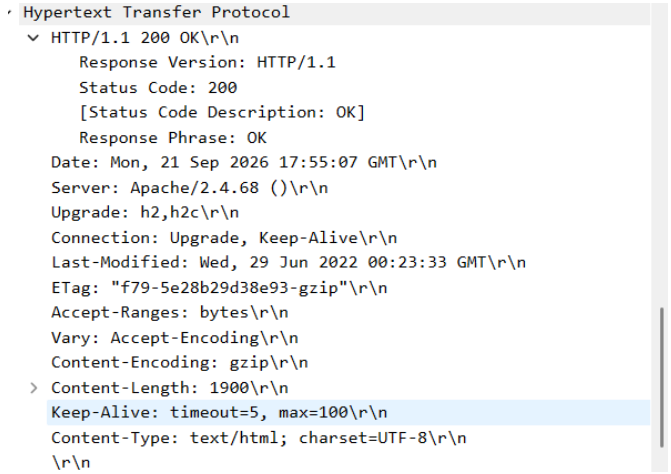


Figure 2: HTTP 200 OK response

Status Code: 200 OK Server: Apache/2.4.68 Content-Type: text/html; charset=UTF-8

The 200 OK response means the server found the page and sent it back successfully as an HTML page.

Part 2: Analyzing TCP/IP Traffic

Task 1: Filter TCP Packets

The previous filter was cleared and "tcp" was typed to focus on TCP packets. A TCP packet belonging to the same HTTP conversation (client port 4834, server port 80) was selected. Right-clicking the packet and choosing Follow -> TCP Stream shows the entire conversation between the client and the server; the same result was also obtained using the equivalent display filter tcp.stream==60, which isolates every packet belonging to this one TCP connection.

Task 2: Analyze TCP Handshake and Investigate Data Transfer and Termination

The packets related to the TCP three-way handshake (SYN, SYN-ACK, ACK) that opens the connection were found and selected, and their sequence and acknowledgment numbers were recorded. After the handshake, the HTTP request and response shown in Part 1 represent the data exchanged between the client and the server over this connection. Finally, the TCP termination packets (FIN, ACK) that close the connection were located. All of these packets are listed in the table below.

Packet No.	Info (Flags)	Seq / Ack	Meaning
3138	4834 -> 80 [SYN]	Seq=0	Client starts the connection
3200	80 -> 4834 [SYN, ACK]	Seq=0, Ack=1	Server acknowledges and responds
3201	4834 -> 80 [ACK]	Seq=1, Ack=1	Client acknowledges - connection established
3685	80 -> 4834 [FIN, ACK]	Seq=428, Ack=2274	Server starts closing the connection
3721	4834 -> 80 [FIN, ACK]	Seq=428, Ack=2275	Client closes its side too
3747	80 -> 4834 [ACK]	Seq=2275, Ack=429	Final acknowledgment - connection closed

The handshake shows the client sending a SYN, the server replying with SYN-ACK, and the client replying with ACK, which matches the sequence and acknowledgment numbers expected for a new TCP connection. After the HTTP request and response were exchanged (data transfer), the connection was closed using two pairs of FIN/ACK packets, one from each side.

Part 3: Capturing and Analyzing UDP Traffic

Task 1: Generate UDP Traffic and Capture Packets

Instead of a separate streaming or VoIP application, a new capture was started while the normal background applications on the laptop (browser tabs, sync and streaming connections that use QUIC, etc.) were generating traffic, since these already produce real UDP traffic. The capture was left running until enough packets had been generated, then it was stopped.

Task 2: Filter and Analyze UDP Packets

In the filter bar, "udp" was typed and applied, which filtered out only the UDP packets from the capture. One UDP packet was selected for analysis, using port 443 (commonly used for encrypted web/streaming traffic over QUIC, which runs on top of UDP) talking to a local port on the laptop:

Source Port: 443 Destination Port: 53786 Length: 306 Checksum: 0xa9b1 UDP payload: 298 bytes

Comparing this to the TCP header analyzed in Part 2, the UDP header only has a few fields (source port, destination port, length, and checksum), while the TCP header also has sequence numbers, acknowledgment numbers, flags, and window size. This shows that the UDP header is much simpler and smaller than the TCP header, since UDP does not need to track connections, order, or acknowledgments - it just sends the data.

Part 4: Comparing TCP and UDP

Task 1: Reliability and Data Integrity

Criterion	TCP or UDP	Reasons
Reliability and Connection Establishment	TCP	TCP sets up a connection first using the three-way handshake (SYN, SYN-ACK, ACK), so both sides are ready before data is sent. It also guarantees delivery through acknowledgments and retransmission of lost packets. UDP does not set up a connection and does not guarantee delivery.
Data Integrity and Ordering	TCP	TCP uses sequence numbers so packets are put back in the correct order, and it resends lost or corrupted data. UDP does not guarantee order or delivery - packets can arrive out of order, be duplicated, or be lost with no retransmission.

Task 2: Use Cases and Performance

	TCP	UDP
Use cases	Web browsing (HTTP/HTTPS), email, file transfer (FTP) - anything that needs reliable, ordered, complete delivery.	Video/audio streaming, VoIP calls, online gaming, DNS - applications where speed matters more than guaranteed delivery.
Performance	Slower / more overhead because of connection setup, acknowledgments, and retransmissions, but reliable.	Faster / less overhead because there is no connection setup or acknowledgments, but delivery is not guaranteed.

Conclusion

This lab showed how Wireshark can be used to capture and analyze HTTP and TCP traffic. HTTP requests and responses were seen in plain text since the connection was not encrypted, and the TCP three-way handshake and termination were identified for the same connection. TCP is reliable and connection-oriented, which is why it is used for HTTP, while UDP is faster but does not guarantee delivery, which is why it is used for things like streaming and gaming.